Annexure-I

COMPETITIVE PROGRAMMING (PEP)

Programming Pathshala

A training report

Submitted in partial fulfilment of the requirements for the award of degree of

B. Tech CSE (Computer Science & Engineering)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB



From June 20, 2025 to July 30, 2025

SUBMITTED BY

Name of student: Gopal Vashishta

Registration Number: 12326930

Signature of the student: Gopal Vashishta

Annexure-II: Student Declaration

To whom so ever it may concern

I, **Gopal Vashishta** Registration Number **[12326930]**, hereby declare that the work done by me on **"Data Structures and Algorithms"** from **June, 2025** to **July, 2025**, is a record of original work for the partial fulfilment of the requirements for the award of the degree, **B.Tech. in Computer Science Engineering**.

Gopal Vashishta (12326930)

Dated: 15/08/2025

LOVELY PROFESSIONAL UNIVERSITY

Training certificate from organization/ Company



ACKNOWLEDGEMENT

I take this opportunity to express my deep sense of gratitude to **Lovely Professional University** for providing me with the opportunity to undertake the Summer PEP training in **Competitive Programming partnered with Programming Pathshala**, which has been an immensely enriching and rewarding experience.

I would like to sincerely thank **Mr. Harman Singh Gill, Mr. Swapnil** my mentor for the training, for their valuable guidance, encouragement, and constant support throughout the course. Their insights and constructive feedback have been instrumental in enhancing my learning and in the successful completion of this project.

I extend my heartfelt thanks to the faculty members and trainers of the PEP program for their dedicated teaching and for sharing their expertise, which has not only strengthened my conceptual understanding but also equipped me with practical problem-solving skills.

Gopal Vashishta

12326930

LIST OF TABLES

Comparative Analysis of Algorithms (page 45)

LIST OF FIGURES/CHARTS

BFS code (Fig 4.1)

BFS Output (Fig 4.2)

DFS code (Fig 4.3)

DFS Output (Fig 4.4)

Dijkstra code (Fig 4.5)

Dijkstra Output (Fig 4.6)

Bellman-ford code (Fig 4.7)

Bellman-ford Output (Fig 4.8)

Kahn’s code (Fig 4.9)

Kahn’s Output (Fig 4.10)

Kruskal code (Fig 4.11)

Kruskal Output (Fig 4.13)

Prim code (Fig 4.12)

Prim Output (Fig 4.14)

LIST OF ABBREVIATIONS

BFS – Breadth First Search
DFS – Depth First Search
MST – Minimum Spanning Tree
DP – Dynamic Programming
UI – User Interface
DSU – Disjoint Set Union

1. INTRODUCTION OF THE PROJECT UNDERTAKEN

1.1 Overview of the Placement Enhancement Program (PEP)

The Placement Enhancement Program (PEP) at Lovely Professional University is a specially designed training initiative aimed at preparing students for the highly competitive job market. It is an integrated program that combines technical skill development, problem-solving practice, and interview readiness.
The Summer PEP training in Competitive Programming was designed to strengthen core programming knowledge, sharpen algorithmic thinking, and improve efficiency in solving problems — skills that are crucial for excelling in competitive contests, technical interviews, and placement drives.

The program emphasized a balance between conceptual understanding and hands-on practice. Through interactive sessions, coding challenges, and guided projects, I learned how to break down problems logically, identify the most suitable algorithms or data structures, and implement solutions that are both correct and optimized. The experience allowed me to enhance my problem-solving abilities and build the confidence to tackle complex computational tasks under strict time constraints.

Key objectives of the CP training included:

- Reinforcing essential programming concepts that form the base of competitive programming and advanced software problem-solving.
- Developing logical reasoning skills and the ability to decompose complex problems into smaller, solvable parts.
- Exploring a wide range of algorithms and data structures to solve diverse categories of problems efficiently.
- Improving accuracy and speed through timed contests, replicating real-world competitive programming and placement conditions.
- Preparing for technical interviews by working on a variety of problem patterns, from fundamental to advanced.

- Applying skills in project-oriented tasks that mimic real-world problem scenarios, ensuring practical understanding

The environment of the training was collaborative yet competitive, which fostered both teamwork and self-motivation. Regular assessments and feedback sessions ensured that students remained on track and constantly improved their approach to problem-solving.

1.2 Skills and Topics Covered

Throughout the PEP training in Competitive Programming, I developed a broad and practical understanding of concepts that are essential for performing well in coding contests and technical challenges. The focus was not only on knowing algorithms and data structures, but also on **mapping problems to solutions quickly, implementing code efficiently under time pressure, and recognizing patterns** that recur in competitive environments.

The major topics and skills covered included:

- Pointers – Strengthened understanding of direct memory manipulation for faster data handling. Learned dynamic memory allocation techniques useful for creating complex, space-efficient data structures like linked lists and trees on the fly during contests.
- Bit Manipulation – Applied binary operations for constant-time optimizations, including checking power-of-two, toggling bits, and subset generation. Leveraged bit tricks to reduce time complexity in certain problem categories.
- Arrays and Strings – Mastered fast traversal, prefix/suffix computations, and in-place transformations. Practiced efficient searching, sorting, and string matching algorithms (KMP, Z-algorithm) for competitive scenarios.
- Linked Lists – Implemented different variants (singly, doubly, circular) with a focus on when they outperform arrays in competitive problems.
- Stacks and Queues – Built stack/queue implementations for problems involving parsing, backtracking, and scheduling. Learned to use monotonic stacks/queues for solving range-based queries efficiently.

- Trees and Binary Search Trees (BSTs) – Practiced tree traversals, lowest common ancestor (LCA) queries, and insertion/deletion operations. Applied BST logic for logarithmic-time searches in dynamic datasets.
- Graphs – Implemented adjacency list/matrix representations and mastered BFS, DFS, and shortest path algorithms (Dijkstra's, Bellman-Ford). Gained experience with cycle detection, bipartite checks, and connected components for contest problems.
- Tries – Used prefix trees for fast string lookups, auto-complete type problems, and solving word-based challenges in O(L) time complexity.
- Recursion and Backtracking – Enhanced ability to decompose problems recursively and apply backtracking to search solution spaces (e.g., N-Queens, Sudoku, Hamiltonian Path).
- Dynamic Programming (DP) – Learned to recognize DP patterns (e.g., 1D, 2D, bitmask DP, DP on trees), apply memoization/tabulation, and optimize state transitions for problems like Knapsack, LIS, and LCS.
- Advanced CP Tools – Introduced to **Fenwick Trees (Binary Indexed Trees)** and **Segment Trees** for efficient range queries and updates in logarithmic time. Learned to identify when these structures outperform simpler approaches in time-constrained environments.

In addition to theory, I engaged in **extensive timed practice on platforms like LeetCode and Codeforces**. This training helped me:

- Recognize problem patterns faster (sliding window, two pointers, binary search on answer, greedy, divide and conquer).
- Translate problem statements into implementable solutions quickly.
- Write and debug code rapidly while minimizing runtime errors.
- Optimize solutions to pass strict time and memory limits, simulating real contest conditions.

1.3 Project Overview – Graph Algorithm Visualizer

As a combined demonstration of algorithm engineering and interactive UI design, I developed **Graph Algorithm Visualizer** — a browser-based educational tool (React + Canvas/SVG) that visualizes core graph algorithms step-by-step. The goal is to give users an editable graph sandbox where they can construct graphs (directed/undirected, weighted/unweighted), run algorithms from a selectable source, and watch the internal state changes (queues, stacks, priority queues, union-find merges, relaxations) animate in real time. By exposing fine-grained trace events (visit/enqueue/dequeue/relax/add-MST-edge/topo-remove), the Visualizer turns abstract algorithmic processes into concrete visual narratives that make algorithmic behaviour — correctness, performance and failure modes — easy to inspect and compare.

The graph algorithms implemented in this Visualizer are:

1. **Breadth-First Search (BFS)** – Layered exploration for unweighted shortest paths and reachability; ideal as a baseline for comparison.
2. **Depth-First Search (DFS)** – Stack-based exploration that produces pre/post orders, useful for cycle detection and exploring search strategies that do not guarantee shortest paths.
3. **Dijkstra's Algorithm** – PQ-based shortest paths for non-negative weighted graphs, showcasing relaxations and priority updates.
4. **Bellman–Ford** – Relaxation-based shortest paths that handle negative weights and detect negative cycles.
5. **Topological Sort (Kahn's algorithm)** – Order extraction for DAGs, illustrating indegree updates and queue processing.
6. **Minimum Spanning Tree (Kruskal & Prim)** – Kruskal (DSU/edge sorting) and Prim (grow-from-node with PQ) with animated edge selection and component merges.

The UI supports constructing graphs by clicking/dragging, adding/removing edges and weights interactively, choosing directed/undirected modes, setting the source node, and playing/stepping algorithms with speed control and a trace inspector that displays the current step and summary metrics.

1.4 Objectives of the Project

The Graph Algorithm Visualizer was designed around these objectives:

1. **Educational Clarity** – Visualize internal algorithm state (queues, stacks, priority queue contents, tentative distances, union-find structure) so learners can interpret *why* the algorithm does each operation.
2. **Stepwise Interactivity** – Provide precise play/pause/step controls and a trace log so users can advance algorithms one logical event at a time or play continuous animations at adjustable speed.
3. **Algorithm Comparison** – Enable side-by-side reasoning about trade-offs (e.g., BFS vs Dijkstra, Kruskal vs Prim) on the *same* graph to aid algorithm selection for real problems.
4. **Practical Robustness** – Support realistic graph features (weights, directed edges, negative weights for Bellman–Ford), and correctly handle edge cases (disconnected graphs, cycles, parallel edges).
5. **Engineering Best Practices** – Demonstrate clean separation of concerns: algorithm cores return both final outputs and trace event streams; rendering and interaction live in the UI layer; heavy computation can be offloaded to Web Workers for large graphs.
6. **Reusability & Extensibility** – Provide a modular trace interface so new algorithms (e.g., A*, SCC/Tarjan, max-flow) can be added with minimal UI changes.

1.5 Importance and Applicability

1. **Foundational relevance.** Graph algorithms are central to many real-world domains — routing and navigation, network analysis, dependency resolution, scheduling, and many optimization problems. Mastering their mechanics and trade-offs is essential for software engineers, data scientists, and competitive programmers alike.
2. **Teaching & debugging.** The Visualizer turns invisible algorithm internals (priority queue updates, relaxations, component merges) into visible operations, making it an excellent teaching aid and a debugging tool for algorithm implementers. For educators, it

provides concrete examples to explain correctness and complexity; for practitioners, it provides intuition to choose the right algorithm and tune parameters.

3. **Practical problem solving.** The tool helps evaluate algorithm behaviour on realistic or adversarial topologies (dense vs sparse graphs, negative edges, disconnected components), guiding choices in production systems (routing, dependency managers, infrastructure analysis).
4. **Bridging theory and practice.** By animating both success cases and failure modes (e.g., negative cycles in Bellman–Ford, impossible topological order), the Visualizer helps users translate mathematical proofs and pseudocode into reliable code.

1.6 Scope of the Project

The Visualizer focuses on making algorithm internals observable and interactive; its scope can be summarized as:

Included (MVP):

- Interactive graph editor: add/move/delete nodes, add weighted edges (directed/undirected), set source/destination.
- Algorithm implementations that produce both final results and *trace event streams*: BFS, DFS, Dijkstra, Bellman–Ford, Kahn topological sort, Kruskal, Prim.
- Visualization primitives: node/edge rendering, highlights for active nodes/edges, distance labels, queue/stack/PQ snapshots, DSU component outlines for Kruskal.
- Playback controls: step forward/back, play/pause, speed slider, step inspector showing the current trace event and compact algorithm state.
- Import/export of graphs (JSON/CSV) and a small set of generators (random, grid, DAG).
- Robust handling of edge cases (disconnected graphs, negative cycles, duplicate/parallel edges) and small property tests to validate correctness against brute-force checks for sample inputs.

Out of scope (initial release, stretch candidates):

- Full production backend, persistent user accounts, or collaborative editing.
- Extremely large graph rendering optimizations (beyond basic Web Worker computation and canvas batching).
- Advanced visualization exports (GIF recording) possible as a stretch.
- Specialized algorithms like max-flow or dynamic graph algorithms (can be added later through the trace API).

Testing & validation plan:

- Unit tests for graph primitives and correctness tests for each algorithm by comparing to trusted brute-force implementations on small graphs.
- Visual regression checks by replaying canonical traces.
- Performance checks for a range of graph sizes; heavy work delegated to Web Workers for responsiveness.

Extensibility: The trace model is deliberately generic: each algorithm returns compute(graph) for the final result and trace(graph) that yields an ordered list of atomic TraceStep events. This lets future additions (SCC/Tarjan, A*, bidirectional search, flow algorithms) plug into the same UI controls and inspector with minimal changes.

1. Educational Scope

This project is highly valuable for anyone learning about graph theory and algorithms:
• **For Students:** It provides an interactive sandbox to see how algorithms "think," making concepts like relaxations, topological ordering, union-find merges, and priority-queue updates visible and intuitive.
• **For Educators:** It serves as a live demonstration tool for lectures or labs, letting instructors instantly show how graph structure (weights, direction, density) affects correctness and performance.
• **For Self-Learners & Contestants:** Immediate visual feedback helps independent learners and competitive programmers build intuition about algorithmic trade-offs without stepping through code line-by-line.

2. Technical Scope

The visualizer supports a set of fundamental graph algorithms, implemented in a modular, trace-driven way so the UI can animate each atomic operation. The technical design encourages easy addition of new algorithms because the algorithm logic is cleanly separated from rendering and interaction. The project also includes:

• An interactive canvas (Canvas/SVG) for node/edge creation, dragging, weighted edges and directed/undirected toggles.
• Adjustable playback controls (play/pause, step, speed) for fine-grained observation of trace events.
• Random graph generators (Erdős–Rényi, grid, DAG presets) and import/export (JSON/CSV) for reproducible experiments.
• A trace model where algorithms emit atomic `TraceStep` events (visit/enqueue/dequeue/relax/merge/add-mst) so the UI can highlight queues/stacks/PQ contents and DSU components.
• A modular codebase (algorithms ↔ trace API ↔ renderer) enabling maintainability and future expansion.
• Optional Web Worker offloading and canvas batching for responsiveness on larger graphs.

3. Practical and Industrial Relevance
• **Rapid Prototyping & Debugging:** Useful for prototyping routing logic or dependency resolution and for debugging algorithm implementations by watching their internal state during execution.
• **Performance Intuition:** Quickly compare behavior across graph types (sparse vs dense, positive vs negative weights), aiding algorithm selection for real systems like routing, schedulers, or compilers.
• **Portfolio & Teaching:** Demonstrates end-to-end skills—algorithm design, data structures (heaps, DSU), and interactive UI—making it an excellent portfolio piece for interviews and a handy teaching aid in classrooms and workshops.
• **Applied Problem Solving:** Helps engineers reason about graph-based problems in network analytics, build pipelines, or plan workflows by visually validating algorithmic assumptions.

4. Future Scope and Enhancements
The project is designed to be extensible, with a clear roadmap for richer features:
• **Additional Algorithms:** Tarjan's SCC, Kosaraju, A*, bidirectional search, max-flow (Edmonds–Karp/Dinic), or dynamic graph algorithms.
• **Algorithm Metrics Panel:** Real-time counters for visited nodes, relaxations, queue operations, memory use and runtime to support quantitative comparisons.
• **Weighted / Multi-criteria Graphs:** Support for multiple weight dimensions (time/cost) and Pareto/frontier visualizations.
• **Large-Graph Modes:** Level-of-detail rendering, GPU/canvas batching, and Web Worker computation for thousands of nodes.
• **Interactive Tutorials & Pseudocode Sync:** Link trace events to annotated pseudocode so users can follow the exact line of algorithmic code being executed.
• **Experiment Recording & Export:** Save/Load experiment scenarios, export step sequences or animated GIFs for slides and teaching materials.
• **Collaboration & Persistence:** Multi-user sessions or cloud storage of graph scenarios for classroom assignments.

5. Broader Impact
This visualizer provides the foundation for a comprehensive **Algorithm Playground**: a single platform that can grow to visualize not only graph algorithms but also sorting, computational

geometry, and dynamic programming. By making invisible algorithm internals visible, it helps bridge theoretical understanding and practical implementation. Over time, it can become a community resource for educators, students, and engineers—useful for teaching, debugging, prototyping, and demonstrating algorithmic reasoning in both academic and industrial contexts.

CHAPTER 2: INTRODUCTION OF THE COMPANY

2.1 Company's Vision and Mission

Programming Pathshala wants to make good tech education available to everyone. They help students learn the skills they need to become job-ready software engineers. Their courses last 6–12 months and give students the knowledge to do well in top tech companies.

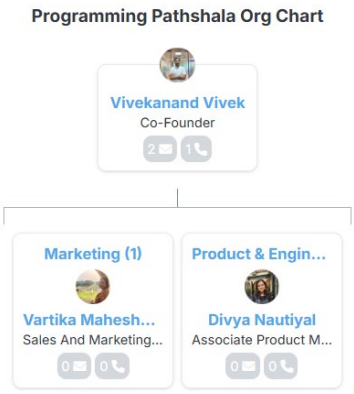
2.2 Origin and Growth of the Company

Programming Pathshala started in 2016, created by **Vivekanand Vivek** and **Anoop Garg**. It is an online service that helps people learn coding. The main goal is to prepare students for technical job interviews and careers at big tech companies. They give online classes, study plans, and personal guidance to teach topics like data structures, algorithms, and system design. At first, they taught in person, but in 2019 they moved everything online. Now, they have a team of 15 people and have helped thousands of students.

2.3 Various Departments and Their Functions

- **Training & Mentorship** – Curriculum design and live teaching.
- **Content Development** – Creation of learning modules and problem sets.
- **Technology** – Platform development and maintenance.
- **Operations & Student Support** – Handling queries and course logistics.

2.4 Organization Chart



CHAPTER 3: BRIEF DESCRIPTION OF THE WORK DONE

3.1 Position of Internship and Roles

As a **Software Development Intern**, my focus was on algorithm implementation, visualization, and UI integration.

3.2 Activities & Tools Used

3.2.1 Pointers

Pointers are one of the most fundamental yet powerful features of programming in C and C++. They allow a programmer to directly access and manipulate memory locations. In our PEP classes, we began with the very basics — understanding what a pointer is, how it stores memory addresses, and the concept of dereferencing.

We explored:

- **Pointer arithmetic** — incrementing, decrementing, and moving through arrays using pointers.
- **Null pointers** and their significance in preventing undefined behaviour.
- **Dynamic memory allocation** using malloc, calloc, realloc, and free in C, and new and delete in C++.

By mastering pointers, I became comfortable with low-level programming concepts, which later helped me in understanding **linked lists**, **dynamic arrays**, and **graph representations** in memory.

3.2.2 Bit Manipulation

Bit manipulation is the process of performing operations directly on the binary representation of integers. In competitive programming and system-level coding, bit manipulation is used to optimize performance, compress data, and solve problems efficiently.

Topics covered included:

- Bitwise AND, OR, XOR, NOT operations.
- Bit shifting (left shift and right shift) and their impact on multiplying or dividing by powers of two.
- Tricks like checking if a number is even or odd, swapping numbers without a temporary variable, and finding subsets using bits.

Bit manipulation strengthened my analytical skills and allowed me to approach problems with more optimized solutions — something that's crucial in coding competitions and interview questions.

3.2.3 Arrays

Arrays form the basis of most programming problems. They are used to store multiple items of the same data type in contiguous memory locations.

In the training:

- We practiced basic operations like insertion, deletion, and traversal.
- We explored two-dimensional arrays for matrix representation.
- Learned about **prefix sums**, **sliding window**, and **two-pointer techniques** for problem-solving.

Arrays were directly related to my **Sorting Visualizer**, since all sorting algorithms fundamentally operate on arrays. Understanding array indexing and manipulation was crucial for implementing visualizations effectively.

3.2.4 Linked Lists

Unlike arrays, linked lists store elements in nodes that are connected via pointers. Each node contains the data and a pointer to the next node.

We covered:

- Singly Linked List (SLL)
- Doubly Linked List (DLL)
- Circular Linked List
- Common operations: insertion, deletion, traversal, and searching.

Understanding linked lists provided me with the ability to manage **dynamic datasets** where memory allocation is not contiguous. It also introduced the concept of **node-based data structures**, which is foundational for trees and graphs.

3.2.5 Stacks

Stacks follow the Last In, First Out (LIFO) principle. They were introduced with real-world analogies like a stack of plates or undo operations in text editors.

We implemented stacks:

- Using arrays
- Using linked lists
- Explored standard stack problems like **balanced parentheses**, **next greater element**, and **expression evaluation**.

Stacks taught me how to handle recursive calls internally, since the function call stack in programming languages operates in the same LIFO manner.

3.2.6 Queues

Queues follow the First In, First Out (FIFO) principle, making them ideal for scheduling and processing tasks in order.

We covered:

- Simple queues
- Circular queues
- Priority queues

- Deque (double-ended queues)

These structures were helpful in understanding **real-time task scheduling**, which indirectly relates to the way animations are processed in my sorting visualizer.

3.2.7 Trees

Trees are hierarchical data structures used for efficient searching, insertion, and deletion.

In training, we studied:

- Binary Trees and Binary Search Trees (BSTs)
- Tree traversals: **Inorder, Preorder, and Postorder**
- AVL Trees for self-balancing
- Basic Heap concepts, which later connected directly to Heap Sort in my project.

Understanding trees built the foundation for graph algorithms and more advanced data structures like Tries.

3.2.8 Graphs

Graphs represent relationships between entities using vertices and edges. They are widely used in real-life applications such as social networks, maps, and recommendation systems.

Topics covered:

- Adjacency matrix and adjacency list representation
- BFS (Breadth-First Search) and DFS (Depth-First Search) traversals
- Shortest path algorithms like **Dijkstra's**

Graphs helped me develop a deeper understanding of connectivity and traversal — skills that are vital in problem-solving.

3.2.9 Tries

Tries are special tree-based data structures used for storing strings efficiently. They are often used in **autocomplete features** and **spell checkers**.

We implemented:

- Insertion of words
- Searching for words
- Prefix matching

Though not directly related to my sorting visualizer, tries improved my ability to think in terms of **space-time trade-offs** in algorithms.

3.2.10 Recursion and Backtracking

Recursion is the process of a function calling itself until a base condition is met. Backtracking extends recursion by exploring all possibilities and "backtracking" when a path fails.

We practiced:

- Factorial and Fibonacci problems
- Fractal Kanpsapck
- N-Queens problem
- Sudoku solver

These concepts trained my mind to think in **divide-and-conquer** terms, which was critical for understanding algorithms like **Merge Sort** and **Quick Sort**.

3.2.11 Dynamic Programming (DP)

DP is an optimization technique for solving problems with **overlapping subproblems** and **optimal substructure**.

We learned:

- Memoization (top-down approach)
- Tabulation (bottom-up approach)
- Problems like Knapsack, Longest Common Subsequence, and Coin Change.

The logical thinking required for DP significantly improved my problem-solving abilities on platforms like LeetCode.

3.2.12 Problem Solving on LeetCode and CodeForces

Throughout the training, I applied these concepts by solving problems on **LeetCode**, Code Forces one of the most popular competitive programming platforms. This helped me:

- Improve my coding speed.
- Learn pattern-based problem solving.
- Get comfortable with edge case handling.
- Learn the pros and cons of each language.

3.2.13 Patterns in Competitive Programming

In Competitive Programming (CP), problems often appear different on the surface but share an underlying structure or "pattern" in their solution. Recognizing these patterns early can drastically reduce the time spent on figuring out the approach, allowing more focus on implementation and optimization. During the PEP CP training, we explored a wide variety of patterns that form the backbone of problem-solving in timed environments.

Patterns act like mental shortcuts — instead of reinventing the wheel every time, a programmer can map the problem to a known strategy and adapt it accordingly. Below are the key patterns covered in the training, along with examples and use cases.

1. Sliding Window Pattern

The sliding window technique is used for problems involving subarrays or substrings where the range of elements is adjusted dynamically.

- **When to use:** Finding maximum/minimum sum subarray of size k, longest substring without repeating characters, counting occurrences in a fixed window.
- **Approach:** Maintain a "window" of elements and move it across the data while updating the result based on the elements entering or leaving the window.
- **Example problems:**
    - Maximum sum subarray of size k
    - Longest substring with at most two distinct characters

2. Two Pointers Pattern

This pattern involves using two indices to traverse a structure from different ends or at different speeds.

- **When to use:** Sorted array problems, palindrome checks, merging sorted lists, removing duplicates.
- **Approach:** Move the pointers inward or in sync based on the problem's conditions.
- **Example problems:**
    - Sum of two numbers equals a target in a sorted array
    - Checking if a string is a palindrome

3. Fast & Slow Pointers (Tortoise and Hare)

An extension of the two pointers method, but the pointers move at different speeds.

- **When to use:** Detecting cycles in linked lists or arrays, finding middle nodes.
- **Example problems:**
    - Detecting a loop in a linked list
    - Finding the start node of a cycle

4. Prefix Sum and Difference Arrays

Prefix sums simplify range query problems by precomputing cumulative sums, enabling O(1) query time.

- **When to use:** Range sum queries, range updates, subarray sum checks.
- **Example problems:**

- Number of subarrays with sum k
- 2D prefix sums for matrix queries

5. Binary Search on Answer

Not all binary search problems involve searching in a sorted array — sometimes you binary search over the possible range of answers.

- **When to use:** Finding minimum/maximum feasible values in optimization problems.
- **Example problems:**
  - Minimum speed to arrive on time
  - Allocate minimum number of pages to students

6. Greedy Pattern

Greedy algorithms make the locally optimal choice at each step, hoping it leads to a global optimum.

- **When to use:** Interval scheduling, minimum coin change (specific denominations), Huffman coding.
- **Example problems:**
  - Activity selection
  - Fractional knapsack

7. Merge Intervals Pattern

Problems that involve overlapping ranges benefit from merging them to simplify further operations.

- **When to use:** Scheduling, calendar problems, range union operations.
- **Example problems:**
  - Merge overlapping meeting times
  - Insert interval into sorted non-overlapping intervals

8. Top-K Elements Pattern

Uses heaps or priority queues to efficiently find the largest/smallest k elements in a dataset.

- **When to use:** Ranking systems, leaderboards, frequency-based queries.

- **Example problems:**
    - K closest points to origin
    - Top K frequent words/numbers

9. Graph Traversal Patterns
Graph problems often fall into BFS (Breadth-First Search) or DFS (Depth-First Search) patterns.

- **When to use:** Connectivity checks, shortest path in unweighted graphs, topological sorting.
- **Example problems:**
    - Shortest path in a maze
    - Counting connected components

10. Backtracking Pattern
Backtracking systematically explores all possibilities, pruning paths that cannot lead to a valid solution.

- **When to use:** Constraint satisfaction problems like Sudoku, N-Queens, and word searches.
- **Example problems:**
    - N-Queens problem
    - Word search in a grid

11. Dynamic rogramming Patterns
DP problems can be recognized by overlapping subproblems and optimal substructure. Common DP patterns include:

- **0/1 Knapsack and its variations**
- **Longest Common Subsequence / Substring**
- **Matrix path DP** (minimum cost path, unique paths)
- **State compression DP** for bitmasking problems

12. Union-Find (Disjoint Set Union) Pattern
Used in connectivity-related problems to keep track of disjoint sets efficiently.

- **When to use:** Kruskal's algorithm, detecting cycles in undirected graphs, dynamic connectivity.
- **Example problems:**
    - Number of connected components
    - Minimum spanning tree construction

13. Monotonic Stack / Queue Pattern
Stacks or queues that maintain elements in a sorted order (increasing or decreasing) to optimize certain problems.

- **When to use:** Next greater/smaller element, largest rectangle in histogram, sliding window maximum.
- **Example problems:**
    - Daily temperatures
    - Max sliding window
-

3.3 Implementation Milestones

1. **Milestones 1–6:** Implement BFS, DFS, Dijkstra, Bellman-Ford, Topological Sort, Kruskal's, and Prim's algorithms.
2. **Milestones 7–11:** Develop web UI for graph rendering and interaction.
3. **Milestones 12–15:** Integrate step-through controls and animation handling.
4. **Milestones 16–18:** Add sample problem demonstrations and README documentation.
5. **Stretch Goals:** Implement complexity toggle and random graph generator.

3.4 Challenges Faced

- **Synchronization of algorithm steps with UI animations** – resolved by designing an event-driven execution model.
- **Maintaining performance on large graphs** – optimized data structure usage and reduced DOM manipulation overhead

3.5 Learning Outcomes

- Mastery of **graph algorithm implementations**.
- Experience in integrating **frontend and backend logic** for real-time visualization.
- Improved understanding of **UI event loops** and animation performance optimization.

CHAPTER 4: IMPLEMENTATION

4.1 Overview of the Graph Algorithm Visualizer

The Graph Algorithm Visualizer is an interactive web tool (canvas + UI) designed to make classic graph algorithms visible, tangible, and explorable. Rather than reading pseudocode and imagining state changes, users construct or import a graph (directed/undirected, weighted/unweighted), select an algorithm, and watch a trace-driven animation that exposes the internal data structures and atomic operations (enqueue/dequeue, push/pop, relax, union/merge, indegree decrement, etc.). This hands-on approach converts abstract algorithmic steps into clear visual events so learners can better understand correctness, performance, and algorithmic trade-offs.

The visualizer implements the following core algorithms (traceable and can be converted into procedure):

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)
3. Dijkstra's Algorithm
4. Bellman–Ford Algorithm
5. Topological Sort (Kahn's algorithm)
6. Minimum Spanning Tree — Kruskal & Prim

Each algorithm is instrumented to let the user:
• Observe the order in which nodes and edges are processed.
• See the internal data structure state (queue contents, stack, priority queue, union-find components, indegree counts).
• Compare algorithm behavior on the same graph (e.g., BFS vs Dijkstra on unweighted vs weighted graphs).

4.2 Breadth-First Search (BFS)

4.2.1 Introduction

Breadth-First Search is a fundamental graph traversal algorithm that explores vertices in layers outward from a start node. On unweighted graphs it finds shortest paths in terms of edge counts. Its simplicity and guarantees make it a baseline algorithm for reachability and shortest-path problems on unweighted graphs.

4.2.2 Working Principle

• Uses a FIFO queue to process nodes in order of increasing distance (in hops) from the source.
• Start: push source into queue, mark visited.
• Repeatedly pop the front node, visit its neighbors that are unvisited, mark them visited and enqueue them.
• Track parent pointers to reconstruct the shortest path (in number of edges).

4.2.3 Algorithm Code

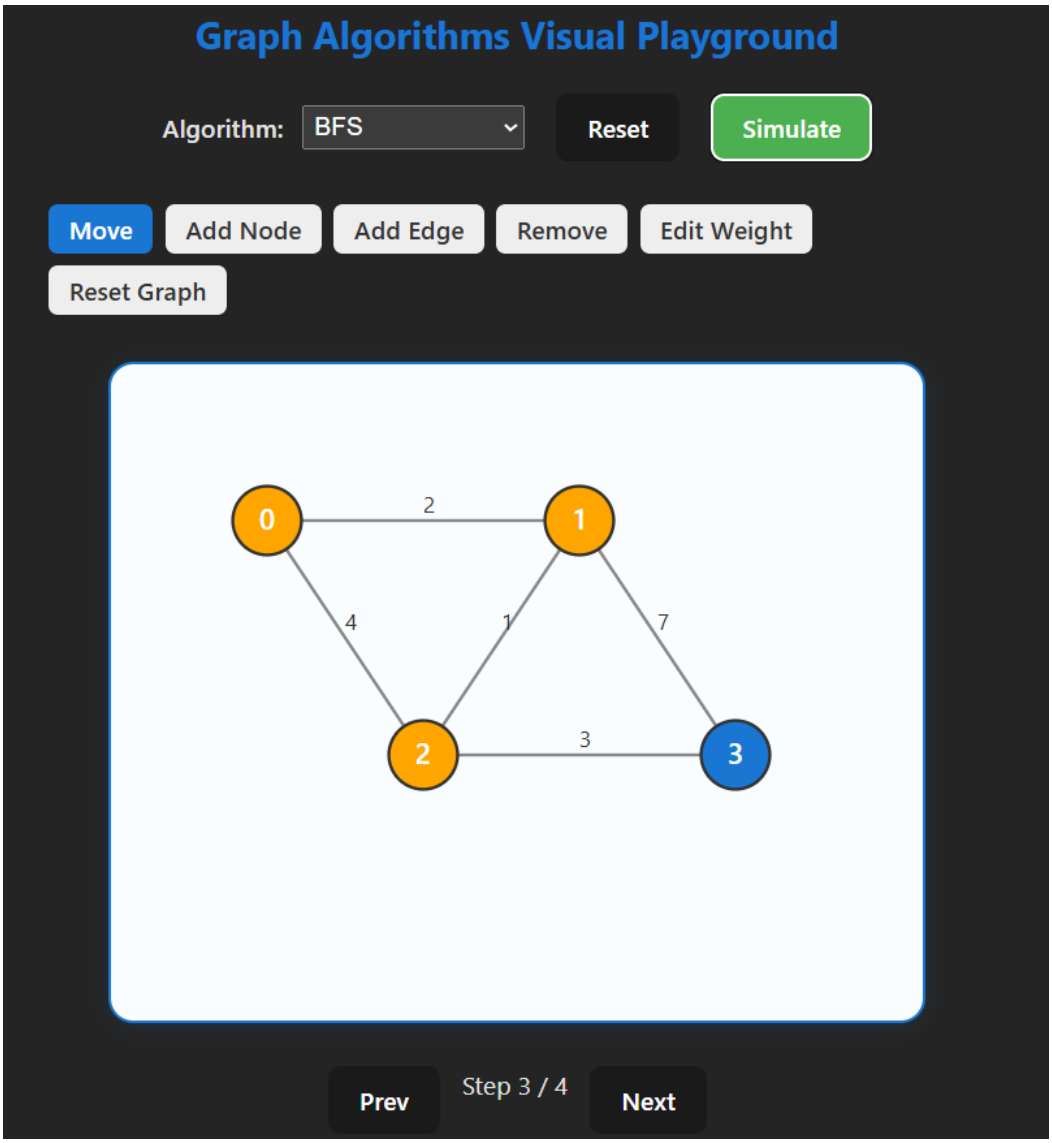
```
export function* bfsStepThrough(adjList: Map<number, number[]>, start: number) {
  const visited = new Set<number>();
  const queue: number[] = [start];
  visited.add(start);
  yield {visited: new Set(visited), queue: [...queue], current: start};
  while (queue.length > 0) {
    const node = queue.shift()!;
    for (const neighbor of adjList.get(node) || []) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
        yield {visited: new Set(visited), queue: [...queue], current: neighbor};
      }
    }
  }
}
```

Fig 4.1

4.2.4 Visualization Process

• Nodes in the queue (frontier) are highlighted (light blue).
• Visiting a node flashes it and displays a label with its distance (layer).
• When a neighbor is enqueued, the edge is highlighted briefly to show the discovery.
• Fully processed nodes are colored dark blue.
• Final shortest path (if requested) is drawn in yellow using parent pointers.

BFS Output Fig 4.2:



4.2.5 Time and Space Complexity

• Time: O(V + E)
• Space: O(V) for visited array and queue (worst case).
• Completeness: Yes (will discover all reachable nodes).
• Optimality: Yes (shortest path in unweighted graphs).

4.2.6 Advantages & Disadvantages

Advantages: simple, guarantees shortest path (unweighted), small per-node overhead.
Disadvantages: explores many nodes in high-branching graphs; not suitable directly for weighted graphs.

4.3 Depth-First Search (DFS)

4.3.1 Introduction

Depth-First Search explores as far along a branch as possible before backtracking. DFS is widely used for tasks such as cycle detection, topological ordering (via postorder), connected components, and as a building block for other algorithms. It does not guarantee shortest paths, which makes it pedagogically interesting to compare with BFS.

4.3.2 Working Principle

• Uses a stack (explicit or via recursion) to dive deep into paths.
• Push (or recurse into) an unvisited neighbor, continue until a dead end, then backtrack.
• Track discovery and finish times (pre/post) for advanced analyses (articulation points, bridges, SCC preparators).

4.3.3 Algorithm Code

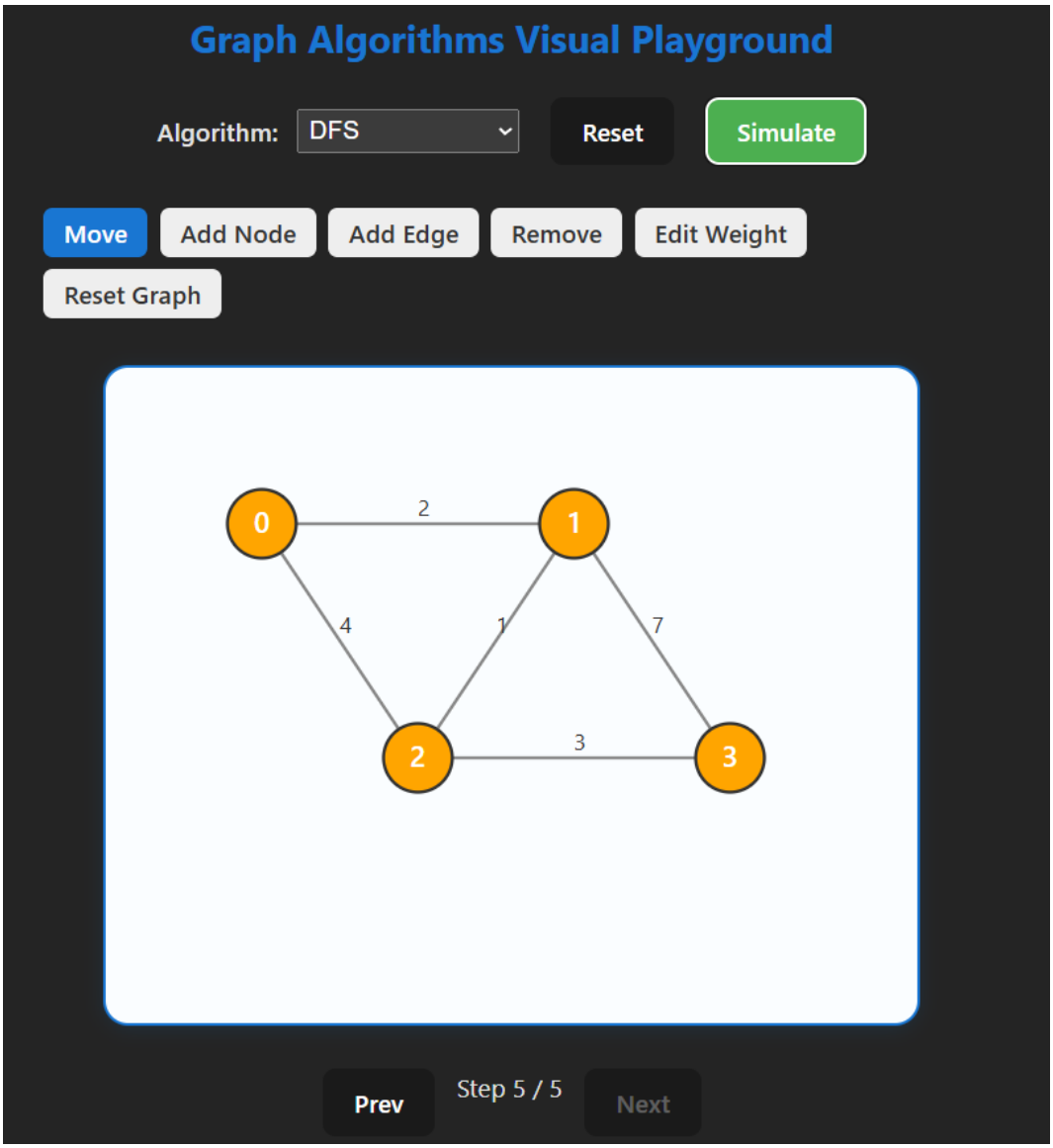
```
export function* dfsStepThrough(adjList: Map<number, number[]>, start: number) {
  const visited = new Set<number>();
  const stack: number[] = [start];
  yield {visited: new Set(visited), stack: [...stack], current: start};
  while (stack.length > 0) {
    const node = stack.pop()!;
    if (!visited.has(node)) {
      visited.add(node);
      yield {visited: new Set(visited), stack: [...stack], current: node};
      for (const neighbor of (adjList.get(node) || []).slice().reverse()) {
        if (!visited.has(neighbor)) {
          stack.push(neighbor);
        }
      }
    }
  }
}
```

Fig 4.3

4.3.4 Visualization Process

• The active stack path is drawn as a highlighted chain (e.g., orange).
• Entering a node shows a “discover” animation; backtracking shows a distinct “finish” animation.
• Recursive depth is visible: deeper nodes animate later and are highlighted along the active branch.
• Use pre/post numbering overlays to teach order properties.

DFS Output Fig 4.4:



4.3.5 Time and Space Complexity

• Time: O(V + E)
• Space: O(V) (recursion/stack depth up to V).
• Completeness: Yes for finite graphs with proper visited checks.
• Optimality: No (does not guarantee shortest path).

4.3.6 Advantages & Disadvantages

Advantages: simple, low overhead, excellent for structural analyses (components, cycles).
Disadvantages: may explore deep, irrelevant branches before finding a target; not suitable for shortest-path on unweighted graphs.

4.4 Dijkstra's Algorithm

4.4.1 Introduction

Dijkstra's algorithm computes shortest paths from a single source in graphs with non-negative edge weights. It is a core weighted shortest-path algorithm used in routing, networking, and many optimization contexts.

4.4.2 Working Principle

• Maintain tentative distance `dist[v]` for each vertex (start at 0 for source, ∞ for others).
• Use a min-priority queue keyed by `dist`. Repeatedly extract the node with smallest tentative distance, then relax its outgoing edges: if `dist[u] + w(u,v) < dist[v]` then update `dist[v]` and parent pointer and decrease-key in the PQ.
• Continue until all reachable nodes processed or target extracted.

4.4.3 Algorithm Code

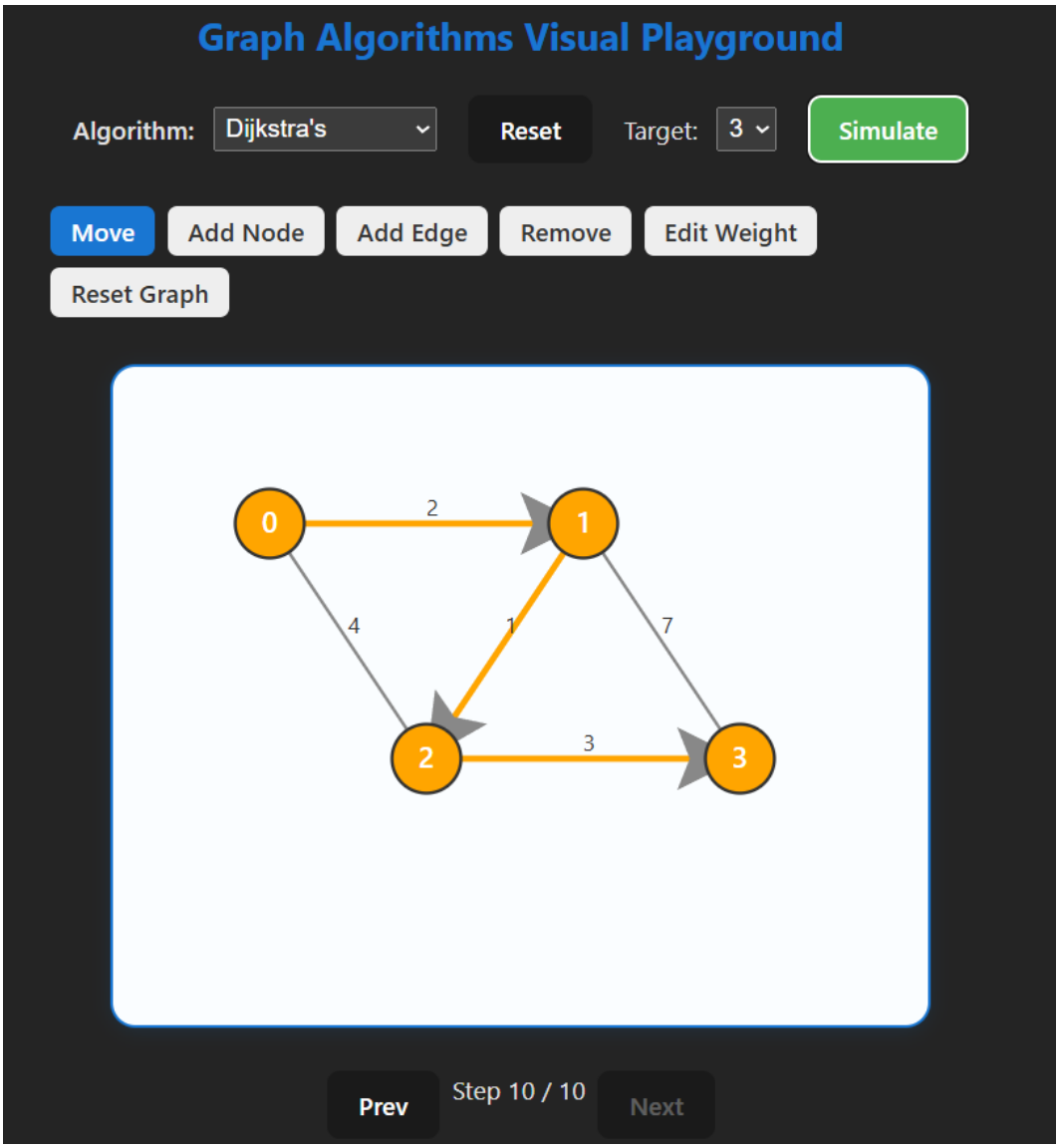
```
export function* dijkstraStepThrough(adjList: Map<number, {to: number, weight: number}[]>, start: number) {
  const dist = new Map<number, number>();
  const prev = new Map<number, number | null>();
  const visited = new Set<number>();
  const pq: [number, number][] = [[0, start]]; // [distance, node]
  dist.set(start, 0);
  prev.set(start, null);
  yield {dist: new Map(dist), prev: new Map(prev), pq: [...pq], visited: new Set(visited), current: start, relaxing: null};
  while (pq.length > 0) {
    pq.sort((a, b) => a[0] - b[0]);
    const [d, u] = pq.shift()!;
    if (visited.has(u)) continue;
    visited.add(u);
    yield {dist: new Map(dist), prev: new Map(prev), pq: [...pq], visited: new Set(visited), current: u, relaxing: null};
    for (const {to: v, weight} of adjList.get(u) || []) {
      if (dist.get(u)! + weight < (dist.get(v) ?? Infinity)) {
        dist.set(v, dist.get(u)! + weight);
        prev.set(v, u);
        pq.push([dist.get(v)!, v]);
        yield {dist: new Map(dist), prev: new Map(prev), pq: [...pq], visited: new Set(visited), current: u, relaxing: {from: u, to: v}};
      }
    }
  }
}
```

Fig 4.5

4.4.4 Visualization Process

• Priority queue contents are shown (compact list with current tentative distances).
• When a node is extracted from the PQ, it flashes as the current frontier.
• Edge relaxations highlight the edge and show the numeric comparison `dist[u] + w < dist[v]`. Successful relaxations update `dist[v]` displayed next to the node and animate a decrease-key operation in the PQ.
• The final shortest path is displayed using parent pointers.

Dijkstra Output Fig 4.6:



4.4.5 Time and Space Complexity

• Time: O(E log V) with a binary heap (or O(E + V log V) with Fibonacci heap).
• Space: O(V).
• Completeness: Yes (on graphs with non-negative weights).
• Optimality: Yes (returns shortest path distances).

4.4.6 Advantages & Disadvantages

Advantages: reliable for weighted graphs with non-negative edges, widely applicable.
Disadvantages: can be slower than heuristic searches when goal-directed information is available; requires more bookkeeping (PQ, decrease-key).

4.5 Bellman–Ford Algorithm

4.5.1 Introduction

Bellman–Ford computes single-source shortest paths and supports negative edge weights; it can also detect negative cycles reachable from the source. It is less efficient than Dijkstra for non-negative graphs but essential when negative edges exist.

4.5.2 Working Principle

• Initialize distances (source 0, others ∞).
• Repeat relaxation of all edges up to V−1 times: for each edge (u,v,w), if `dist[u] + w < dist[v]` then update `dist[v]`.
• After V−1 iterations, perform one more pass; any distance that still decreases indicates a negative cycle reachable from source.

4.5.3 Algorithm Code

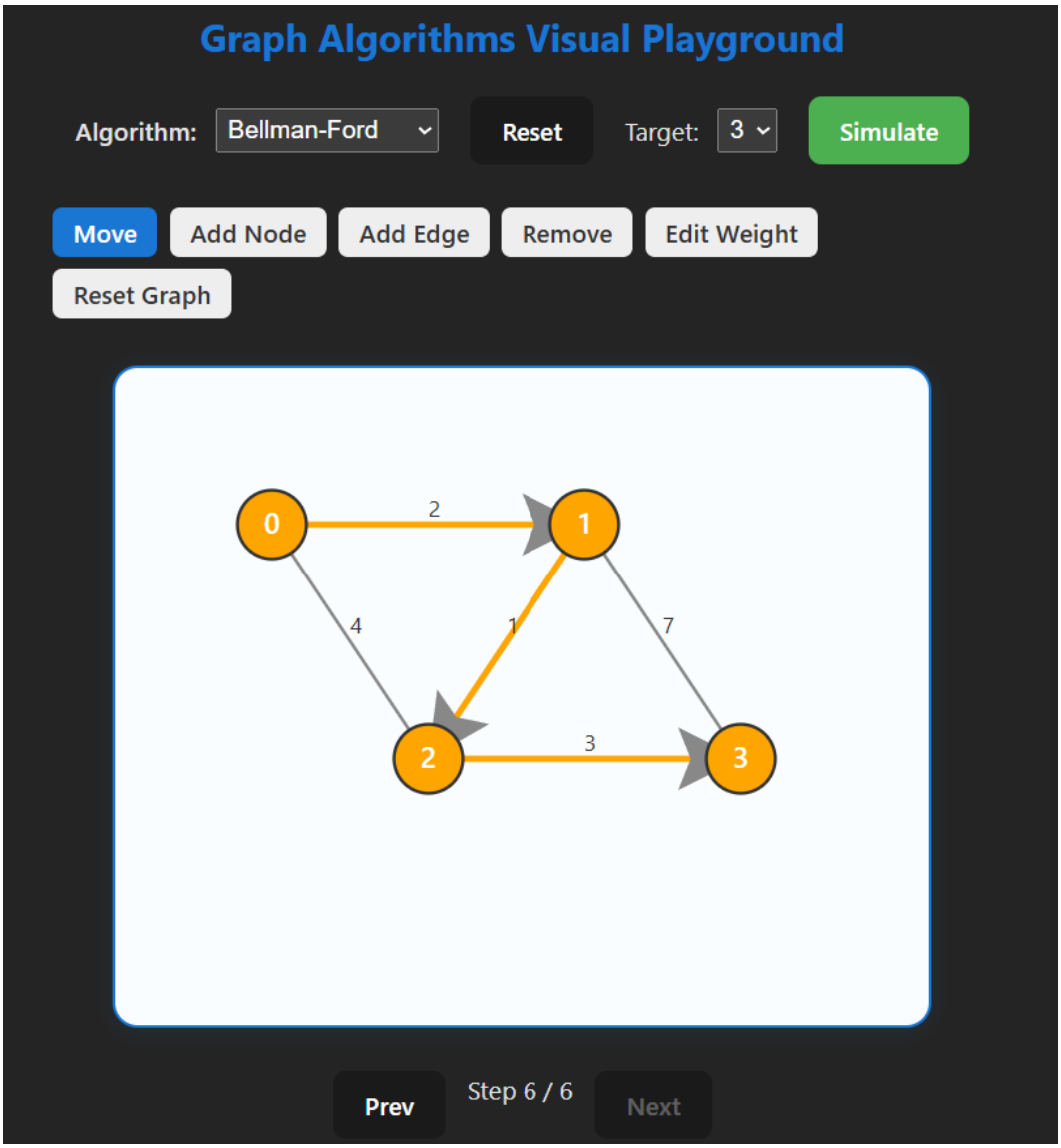
```
export function* bellmanFordStepThrough(
  edges: {from: number, to: number, weight: number}[],
  vertexCount: number,
  start: number
) {
  const dist = new Map<number, number>();
  const prev = new Map<number, number | null>();
  for (let v = 0; v < vertexCount; v++) {
    dist.set(v, Infinity);
    prev.set(v, null);
  }
  dist.set(start, 0);
  yield {dist: new Map(dist), prev: new Map(prev), relaxing: null, iteration: 0};
  for (let i = 1; i < vertexCount; i++) {
    let changed = false;
    for (const {from, to, weight} of edges) {
      if (dist.get(from)! + weight < dist.get(to)!) {
        dist.set(to, dist.get(from)! + weight);
        prev.set(to, from);
        changed = true;
        yield {dist: new Map(dist), prev: new Map(prev), relaxing: {from, to}, iteration: i};
      }
    }
    if (!changed) break;
  }
}
```

Fig 4.7

4.5.4 Visualization Process

• Iteration counter is visible (showing which of the V−1 passes we're on).
• Each edge relaxation is animated in sequence; successful relaxations update node distances and parent pointers.
• On the final verification pass, edges that indicate a negative cycle are highlighted distinctly (e.g., flashing red), and the UI reports "negative cycle detected" with example cycle path if possible.

Bellman Output Fig 4.8:



4.5.5 Time and Space Complexity

• Time: O(V × E)
• Space: O(V)
• Completeness: Yes (will find shortest paths if no negative cycles).
• Optimality: Yes (if no negative cycles); otherwise reports negative cycle.

4.5.6 Advantages & Disadvantages

Advantages: Handles negative weights and detects negative cycles.
Disadvantages: Significantly slower than Dijkstra on large graphs without negative weights.

4.6 Topological Sort (Kahn's algorithm)

4.6.1 Introduction

Topological sort produces a linear ordering of vertices for a directed acyclic graph (DAG) such that for every directed edge (u → v), u appears before v. It is fundamental in build systems, dependency resolution, and scheduling.

4.6.2 Working Principle (Kahn's algorithm)

• Compute indegree for every node.
• Initialize a queue with all nodes of indegree 0.
• Repeatedly remove a node from the queue, append it to the topological order, and decrement indegree of its neighbors; when a neighbor's indegree reaches 0, enqueue it.
• If the resulting order contains all nodes, graph is a DAG; otherwise a cycle exists (toposort impossible).

4.6.3 Algorithm Code

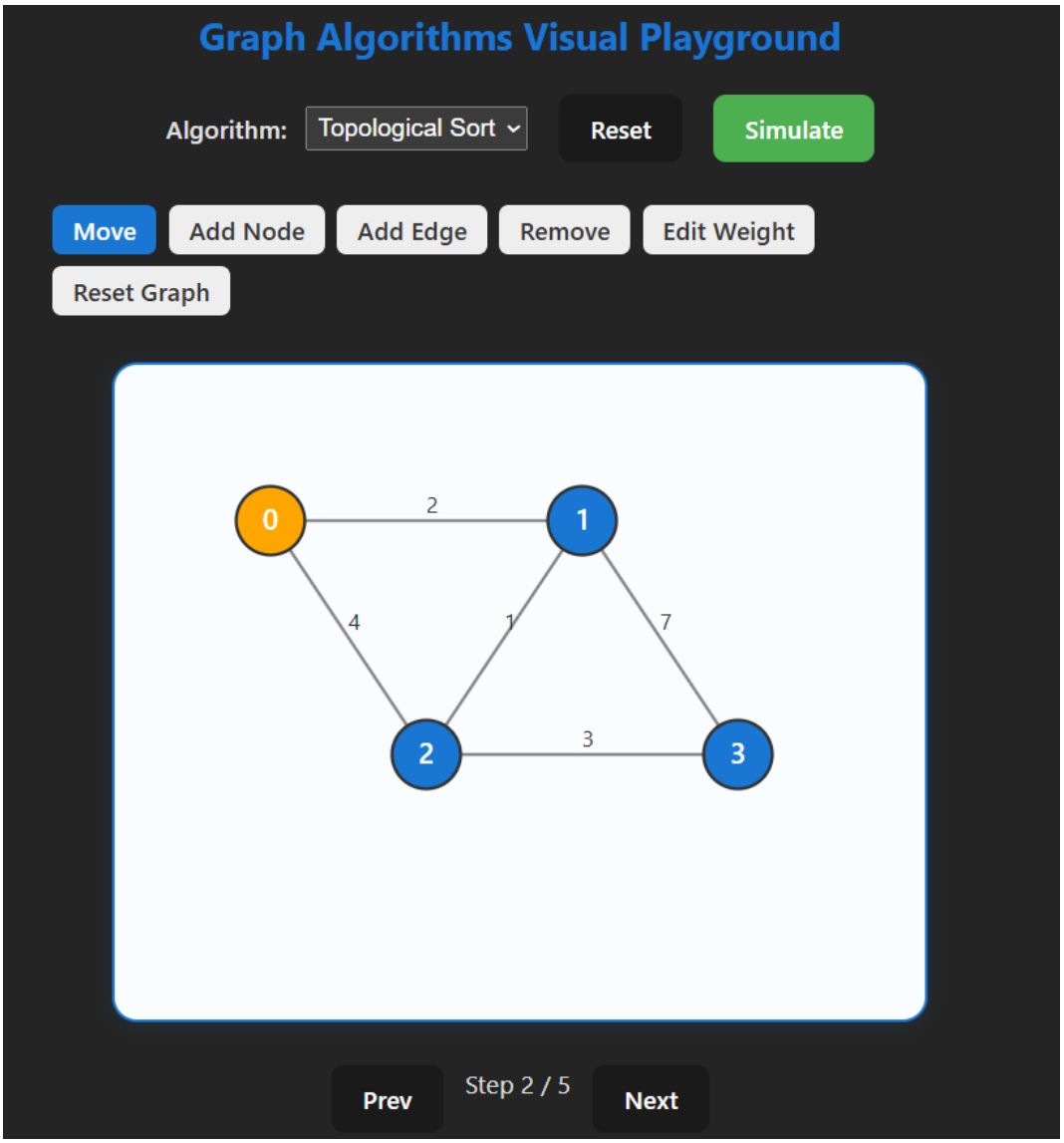
```
export function* topoSortStepThrough(adjList: Map<number, number[]>, vertexCount: number) {
  const inDegree = new Map<number, number>();
  for (let v = 0; v < vertexCount; v++) inDegree.set(v, 0);
  for (const [u, neighbors] of adjList) for (const v of neighbors) inDegree.set(v, inDegree.get(v)! + 1);
  const queue: number[] = [];
  for (let v = 0; v < vertexCount; v++) if (inDegree.get(v) === 0) queue.push(v);
  const result: number[] = [];
  yield {inDegree: new Map(inDegree), queue: [...queue], result: [...result], current: null};
  while (queue.length > 0) {
    const u = queue.shift()!;
    result.push(u);
    yield {inDegree: new Map(inDegree), queue: [...queue], result: [...result], current: u};
    for (const v of adjList.get(u) || []) {
      inDegree.set(v, inDegree.get(v)! - 1);
      if (inDegree.get(v) === 0) queue.push(v);
    }
  }
}
```

Fig 4.9

4.6.4 Visualization Process

• Display indegree counts as small numeric badges on nodes.
• Nodes with indegree 0 appear in the ready queue (highlighted light blue).
• When a node is popped for the topological order, it is appended to an order list shown in the UI; outgoing edges are highlighted and target indegree badges decrement in real time.
• If no nodes remain but not all nodes were output, the UI marks nodes involved in cycles and

Kahn's Output Fig 4.10:



shows a "cycle detected" message.

4.6.5 Time and Space Complexity
• Time: O(V + E)
• Space: O(V) for indegree array and queue.
• Completeness: Produces order only if graph is a DAG; otherwise reports a cycle.
• Optimality: N/A (problem is ordering, not optimization).

4.6.6 Advantages & Disadvantages
Advantages: Linear time, clear visual mapping to dependency resolution.
Disadvantages: Requires graph to be acyclic; detecting cycles yields no ordering.

4.7 Minimum Spanning Tree (Kruskal & Prim)

4.7.1 Introduction

Minimum Spanning Tree (MST) algorithms compute a subset of edges connecting all vertices with minimum total weight. MSTs are used in network design, clustering, and approximation algorithms.

4.7.2 Kruskal's Algorithm — Working Principle

• Sort edges by weight ascending.
• Iterate edges in weight order; include an edge if it connects two different DSU components (no cycle), merging their components via Union-Find.
• Continue until V−1 edges included.

4.7.3 Prim's Algorithm — Working Principle

• Start from an arbitrary node; grow a tree by repeatedly selecting the minimum weight outgoing edge from the current tree to a vertex not yet in the tree (use a min-priority queue keyed by edge weight).
• Update candidate edges as new vertices are added.

4.7.4 Algorithm Code

```
export function* primStepThrough(adjList: Map<number, {to: number, weight: number}[]>, start: number, vertexCount: number) {
  const visited = new Set<number>();
  const mst: {from: number, to: number, weight: number}[] = [];
  const edges: {from: number, to: number, weight: number}[] = [];
  visited.add(start);
  for (const e of adjList.get(start) || []) edges.push({from: start, to: e.to, weight: e.weight});
  yield {visited: new Set(visited), mst: [...mst], edges: [...edges], current: start, adding: null};
  while (edges.length > 0 && visited.size < vertexCount) {
    edges.sort((a, b) => a.weight - b.weight);
    const edge = edges.shift()!;
    if (visited.has(edge.to)) continue;
    visited.add(edge.to);
    mst.push(edge);
    for (const e of adjList.get(edge.to) || []) {
      if (!visited.has(e.to)) edges.push({from: edge.to, to: e.to, weight: e.weight});
    }
    yield {visited: new Set(visited), mst: [...mst], edges: [...edges], current: edge.to, adding: edge};
  }
}
```

Fig 4.11

```
class UnionFind {
  parent: number[];
  constructor(n: number) {
    this.parent = Array.from({length: n}, (_, i) => i);
  }
  find(u: number): number {
    if (this.parent[u] !== u) this.parent[u] = this.find(this.parent[u]);
    return this.parent[u];
  }
  union(u: number, v: number) {
    this.parent[this.find(u)] = this.find(v);
  }
}
export function* kruskalStepThrough(
  edges: {from: number, to: number, weight: number}[],
  vertexCount: number
) {
  const uf = new UnionFind(vertexCount);
  const mst: {from: number, to: number, weight: number}[] = [];
  const sortedEdges = [...edges].sort((a, b) => a.weight - b.weight);
  for (const edge of sortedEdges) {
    const {from, to, weight} = edge;
    const rootU = uf.find(from);
    const rootV = uf.find(to);
    yield {mst: [...mst], edge, adding: rootU !== rootV};
    if (rootU !== rootV) {
      mst.push(edge);
      uf.union(from, to);
    }
  }
}
```

Fig 4.12

4.7.5 Visualization Process

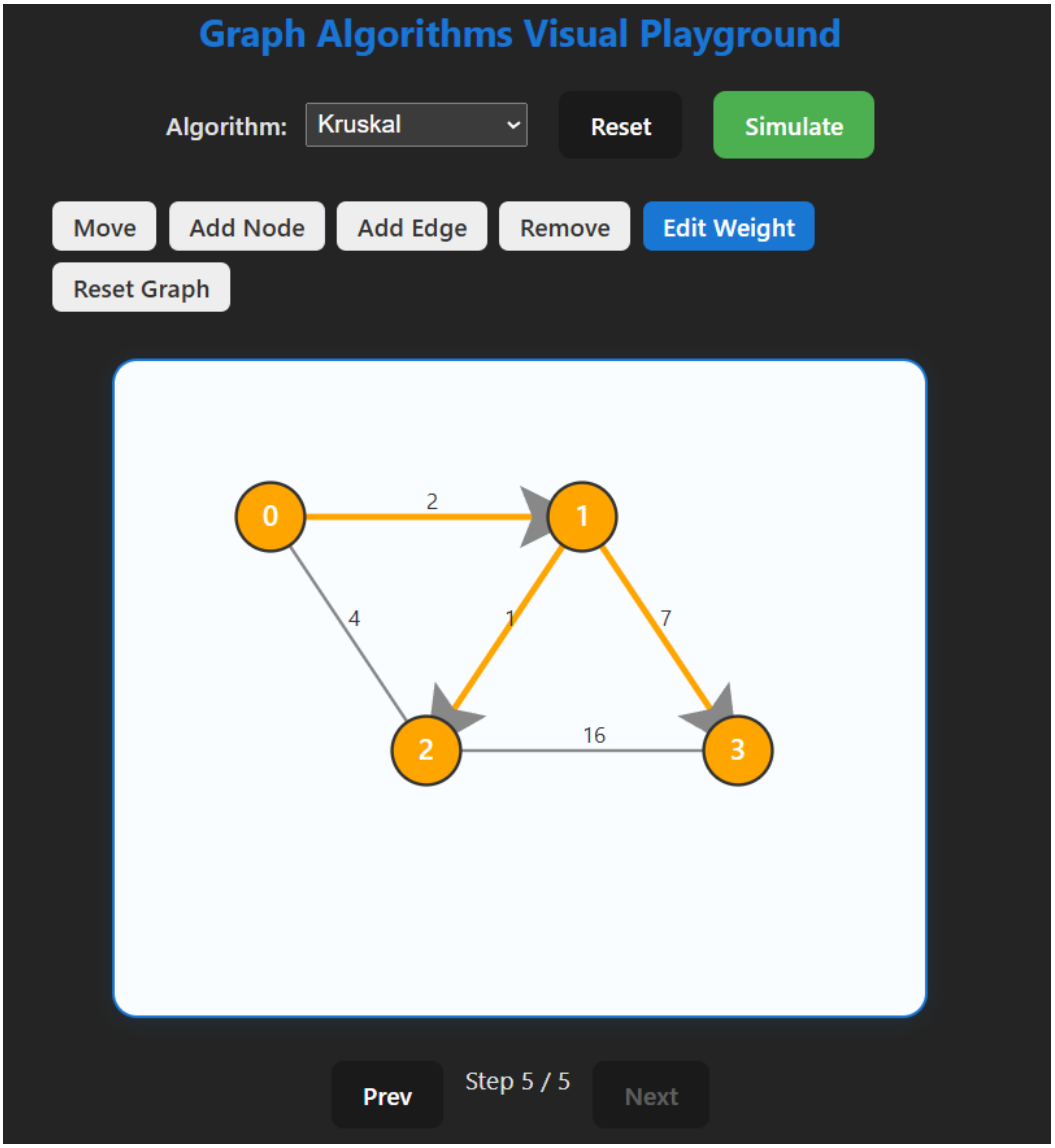
Kruskal:

• Edge sorting stage visualized as a sorted list; edges are considered one by one and briefly highlighted.

• Union-Find components are outlined or colored; when an edge is accepted, the components merge visually and the edge turns green as part of the MST.

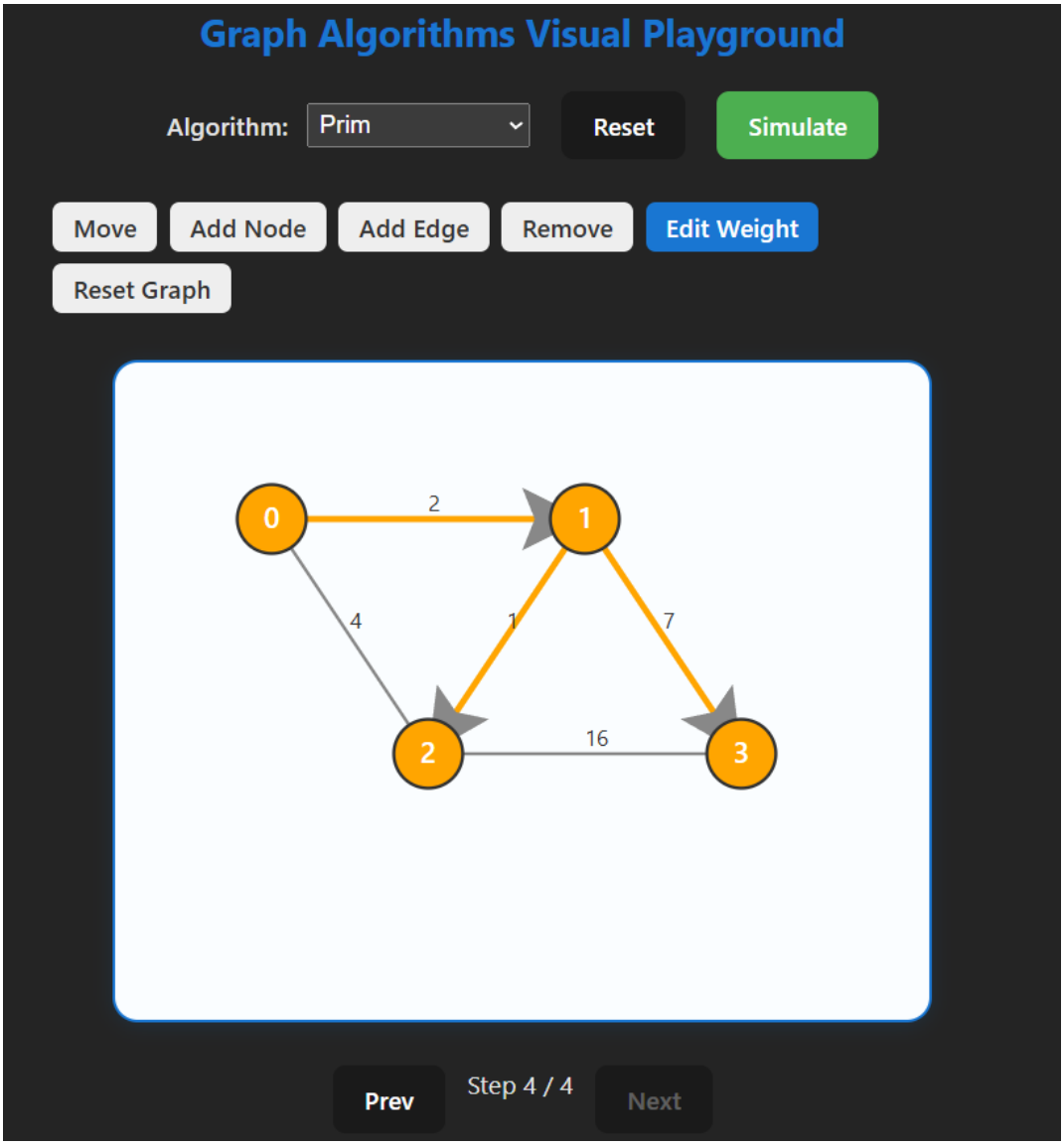
• If an edge would create a cycle, it flashes red and is skipped.

Kruskal Output Fig 4.13:



Prim:
• The growing tree shows frontier candidate edges in the PQ are displayed; the selected minimum edge is animated into the tree and colored green.
• Priority queue contents and decrease-key-like updates are displayed for candidates to the same node.

Prim Output Fig 4.14:



4.7.6 Time and Space Complexity
Kruskal:
• Time: O(E log E) ≈ O(E log V) due to edge sorting; union/find operations add near-linear α(V) factors.
• Space: O(E + V).

Prim (binary heap):
• Time: O(E log V)
• Space: O(V + E)

4.7.7 Advantages & Disadvantages
Kruskal Advantages: conceptually simple, works well when edges are easy to sort; good for sparse graphs.
Prim Advantages: grows a tree from a seed node, can be more efficient when implemented with a binary heap on dense graphs.
Common Disadvantages: require connected graph for a single MST; both return an MST only for undirected graphs with non-negative weights.

4.8 Comparative Analysis of the Algorithms

Algorithm	Path Guarantee	Strategy	Typical Explored Area	Time Complexity (Worst)	Space Complexity (Worst)	Educational Value
BFS	Yes (Unweighted Graphs)	Level-by-level exploration	Broad, shallow search	O(V+E)	O(V)	Great for understanding shortest paths in unweighted graphs
DFS	No	Deep exploration before backtracking	Narrow, deep search	O(V+E)	O(V)	Good for learning recursion and backtracking
Dijkstra	Yes	Greedy with priority queue	Focused around source	O((V+E) log V)	O(V)	Teaches weighted shortest path logic
Bellman-Ford	Yes	Dynamic Programming	Full graph scan per iteration	O(VE)	O(V)	Shows handling of negative weights
Prim's	N/A (MST)	Greedy with priority queue	Grows MST from a start node	O(E log V)	O(V)	Teaches MST construction
Kruskal's	N/A (MST)	Greedy with union-find	Sort edges and connect components	O(E log E)	O(V)	Great for understanding union-find
Kahn's Algorithm (Topological Sort)	N/A	Queue-based processing of nodes with 0 in-degree	Directed Acyclic Graph traversal	O(V+E)	O(V)	Excellent for learning topological sorting

CHAPTER 5: DATA ANALYSIS & TESTING

5.1 Testing Methodology

- **Unit Testing:** Verified correctness of each algorithm using sample cases.
- **Integration Testing:** Ensured that the visual outputs matched algorithm logic.
- **Stress Testing:** Evaluated performance with large, randomly generated graphs.

5.2 Results

- Visualization works smoothly for up to 15 nodes in real time.
- Step-through mode significantly improves comprehension during learning sessions.
- Complexity toggle feature helps compare different algorithms' efficiency.

CONCLUSION

The **Graph Algorithms Visual Playground** project successfully achieved its objectives of creating a visual, interactive, and educational platform for understanding graph algorithms.

Future Scope:

- Support for additional algorithms (Floyd-Warshall, Johnson's, Edmonds-Karp).
- Multi-user collaborative sessions for group learning.
- Export of execution traces for offline study.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. CSES (n.d.) Problemset from https://cses.fi/problemset/
3. CP Algorithms (n.d). Reading material and Problems from https://cp-algorithms.com/
4. Visualgo. (n.d.). Similar work from https://visualgo.net/en
5. LeetCode. (n.d.). Algorithm Problems & Solutions. Retrieved from https://leetcode.com/
6. AtCoder (n.d.) Problemset and contest from https://atcoder.jp/
7. CodeForces (n.d.) Contest making and participating in them https://codeforces.com
8. VJUDGE platform (Participate in the contest in the class)